ChatApp - React Native Real-Time Messaging Platform

ChatApp is a full-stack real-time chat application built with an Expo React Native mobile client, an enterprise-style Node.js backend, Socket.io realtime communication, MySQL persistence, and Redis-backed caching/presence.

The project was built as a practical messaging system with direct chats, group chats, message states, image uploads, reply/forward support, profile management, dark/light mode, localization, and a mobile-first chat UI.

Project Structure

‘‘‘text
ChatApp/
backend/
database/
schema.sql
migrations/
scripts/
archiveOldMessages.js
seedUsers.js
verifyRealtime.js
src/
app.js
config/
controllers/
database/
exceptions/
middleware/
models/
repository/
routes/
services/
socket/
utils/
tests/
server.js
frontend/
src/
components/
context/
navigation/
screens/
services/
styles/
utils/
App.js
app.json

docker-compose.yml
package.json
‘‘‘

Technology Stack

Frontend

- Expo React Native
- React 19
- React Native 0.86
- React Navigation bottom tabs and stack navigation
- Context API for auth, chat, socket, theme, localization, and notifications
- Axios for HTTP API requests
- Socket.io client for realtime messaging
- AsyncStorage for persisted local preferences/session data
- Expo Image Picker for avatar and message image uploads
- Expo Clipboard for copying messages
- Expo Blur for blurred overlays
- Expo Vector Icons for UI iconography

Backend

- Node.js
- Express.js
- Socket.io
- MySQL 8 with mysql2/promise
- Redis 7 for cache, presence, rate limiting, and temporary realtime state
- JWT access tokens and refresh tokens
- bcrypt password hashing
- multer for local file uploads
- express-validator for request validation
- express-rate-limit with Redis-aware application rate limiting
- Node built-in test runner

Infrastructure

- Docker Compose for MySQL, Redis, and phpMyAdmin
- MySQL initialized from backend/database/schema.sql
- Local uploads served from backend/uploads

Current Docker Services

docker-compose.yml currently starts infrastructure only:

- MySQL on host port 3307
- Redis on host port 6379
- phpMyAdmin on host port 8080

The backend and frontend currently run from npm scripts during development.

Local Setup

1. Start Infrastructure

‘‘‘bash
docker compose up -d
‘‘‘

This starts MySQL, Redis, and phpMyAdmin.

2. Install Dependencies

From the project root:

‘‘‘bash
npm install
cd backend && npm install
cd ../frontend && npm install
‘‘‘

3. Configure Environment Files

Create backend env:

‘‘‘bash
copy backend\.env.example backend\.env
‘‘‘

Create frontend env:

‘‘‘bash
copy frontend\.env.example frontend\.env
‘‘‘

For Android emulator, localhost can usually work for Expo web/dev tooling, but Android emulator API access often needs:

‘‘‘text
EXPO_PUBLIC_API_BASE_URL=http://10.0.2.2:5000
EXPO_PUBLIC_SOCKET_IO_URL=http://10.0.2.2:5000
‘‘‘

For a physical phone, use your computer LAN IP:

‘‘‘text

EXPO_PUBLIC_API_BASE_URL=http://YOUR_LAN_IP:5000
EXPO_PUBLIC_SOCKET_IO_URL=http://YOUR_LAN_IP:5000
‘‘‘

4. Generate Secrets

Update backend/.env with strong values:

‘‘‘text
JWT_SECRET=your_long_random_secret
REFRESH_TOKEN_SECRET=your_other_long_random_secret
‘‘‘

5. Run Backend

From the project root:

‘‘‘bash
npm run backend:dev
‘‘‘

Or from backend/:

‘‘‘bash
npm run dev
‘‘‘

6. Run Frontend

From the project root:

‘‘‘bash
npm run frontend:start
‘‘‘

Or from frontend/:

‘‘‘bash
npm start
‘‘‘

Android emulator:

‘‘‘bash
npm run frontend:android
‘‘‘

7. Open phpMyAdmin

Open:

‘‘‘text
http://localhost:8080
‘‘‘

Database:

‘‘‘text
chat_app
‘‘‘

Available Scripts

Root:

- npm run backend:dev starts the backend with nodemon.
- npm run backend:start starts the backend with Node.
- npm run frontend:start starts Expo.
- npm run frontend:android starts Expo for Android.
- npm run frontend:ios starts Expo for iOS.
- npm run frontend:web starts Expo web.

Backend:

- npm run dev starts the API in development mode.
- npm start starts the API in production-like mode.
- npm test runs backend tests with Node test runner.
- npm run seed:users inserts 10 demo users.
- npm run archive:messages archives old messages into the partitioned archive table.

Frontend:

- npm start starts Expo.
- npm run android starts Android target.
- npm run ios starts iOS target.
- npm run web starts web target.

Demo Seed Users

The backend includes backend/scripts/seedUsers.js, which inserts 10 demo users with random human-readable names and test emails.

Run:

‘‘‘bash
cd backend

```
npm run seed:users
```

Default seeded password:

```text
Password@123
```

The script is idempotent by email, so running it again skips existing users.

Backend Architecture

The backend follows an enterprise-style layered architecture:

- Routes define HTTP endpoints and validators.
- Controllers handle request/response orchestration.
- Services contain business logic and transaction flow.
- Repositories contain database access logic.
- Models/entities normalize response shapes.
- Middleware handles auth, validation, logging, rate limiting, sanitization, file uploads, and errors.
- Socket handlers/managers handle realtime events and room membership.
- Utilities centralize tokens, responses, logging, and encryption helpers.
- Exceptions standardize API error behavior.

Core Backend Flow

1. server.js loads env variables, creates the Express app, creates the HTTP server, attaches Socket.io, then starts listening.
2. src/app.js builds the database adapter, repositories, services, controllers, middleware, and routes.
3. A client request enters Express through CORS, JSON parsing, logging, sanitization, and rate limiting.
4. Protected routes use JWT auth middleware.
5. Route validators check payload shape.
6. Controllers call service methods.
7. Services validate business rules and call repositories.
8. Repositories execute MySQL queries through the database adapter.
9. Services emit realtime socket events when persisted state changes.
10. Controllers return standardized success/error response envelopes.
11. Errors pass through centralized error handling.

Database Abstraction

Database access is isolated behind:

- DatabaseAdapter

- MySqlDatabaseAdapter
- repository classes

This keeps raw SQL away from controllers and most services, making future database migration easier. Business logic depends on repository methods instead of direct database calls.

Base Classes

The backend includes reusable base classes:

- BaseRepository for generic CRUD operations.
- BaseService for shared service behavior.
- BaseController for controller helpers and async handling.

Concrete repositories/services/controllers extend these patterns for application-specific behavior.

Backend Packages

- bcryptjs: password hashing.
- cors: CORS middleware.
- dotenv: env file loading.
- express: HTTP API framework.
- express-rate-limit: rate limiting.
- express-validator: request validation.
- jsonwebtoken: JWT access/refresh tokens.
- multer: multipart image uploads.
- mysql2: MySQL promise client.
- redis: Redis client.
- socket.io: realtime server.
- nodemon: development auto-restart.

Frontend Architecture

The frontend is organized around reusable components, screens, navigation, service clients, and context providers.

Frontend Flow

1. App.js loads providers.
2. RootNavigator decides whether to show auth or app navigation.
3. Auth screens call authApi.
4. Tokens/session data are stored through AuthContext.
5. App screens use ChatContext for conversations, messages, users, and optimistic updates.
6. Socket connection is managed through the socket service/context.
7. Incoming socket events update chat state and show toast notifications when appropriate.
8. Theme and language changes are stored locally and applied across screens.

Frontend Packages

- @expo/vector-icons: Ionicons and FontAwesome icons.
- @react-native-async-storage/async-storage: local session/preferences storage.
- @react-navigation/bottom-tabs: bottom tab navigator.
- @react-navigation/native: navigation foundation.
- @react-navigation/stack: stack navigation and transitions.
- axios: backend API client.
- babel-preset-expo: Expo Babel setup.
- expo: Expo runtime.
- expo-blur: blur overlays.
- expo-clipboard: copy message text/image URL.
- expo-image-picker: avatar/group/message image selection.
- expo-status-bar: status bar integration.
- react, react-native: mobile UI framework.
- react-native-gesture-handler: gestures/navigation support.
- react-native-safe-area-context: safe-area handling.
- react-native-screens: native navigation screens.
- socket.io-client: realtime client.

Main Features

Authentication

- Register
- Login
- Refresh token flow
- Logout
- JWT verification
- Password hashing with bcrypt
- Styled login/register/password reset screens
- Social login buttons as UI placeholders
- Show/hide password icon

Conversations

- Direct one-to-one conversations
- Group conversations
- Conversation list
- Group list tab
- People tab
- Search screen
- User search screen
- Archive, mute, pin, delete, leave conversation
- Pinned conversations appear at the top
- Muted conversations show mute badge
- Group owner role support
- Group profile page

- Group avatar and title update for owner
- Add/kick group members for owner

Messaging

- Text messages
- Image messages from device gallery
- Fullscreen image viewer
- Optimistic message sending
- Pending/failed/retry state support
- Delivered/read state support
- Read receipts
- Unread counts
- Cursor-based history loading
- Load older messages
- Message edit
- Message delete
- Message copy
- Reply to message
- Forward message
- Multi-select forwarding to multiple users/conversations
- Message search

Realtime

- Socket.io connection after authentication
- User rooms
- Conversation rooms
- Online/offline presence
- Active now indicator
- Animated online green dot
- Offline dot style
- Typing indicator
- Message received events
- Message edited/deleted events
- Delivered/read events
- Realtime toast notifications

Attachments

- Avatar upload
- Group avatar upload
- Chat image upload
- Local upload storage through backend
- Uploaded files served from /uploads
- Attachment metadata stored separately from messages

Notifications

- Toast notification system
- Draggable/dismissible toast behavior
- Message notification toast with avatar, name, time, and preview
- Notification settings page
- Mute all notifications
- Per-user notification mute
- Device token table prepared for future push notifications

UI/UX

- Mobile-first chat UI inspired by Messenger/Telegram patterns
- Custom bottom tab navigator
- Smooth active tab indicator
- Floating/glass-style tab bar
- Chat list MyDay/story row
- Collapsible MyDay behavior
- Keyboard-aware chat composer
- Long-press action sheets for messages and conversations
- Reusable confirmation modal
- Dedicated language selection page
- English and Myanmar localization
- UK/Myanmar flag icons
- Dark/light mode
- Reusable animated switch component

API Overview

Base API prefix:

‘‘‘text
/api/v1
‘‘‘

Auth:

- POST /auth/register
- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- GET /auth/verify

Users:

- User profile read/update
- Avatar upload
- User listing/search

Conversations:

- GET /conversations
- POST /conversations/direct
- POST /conversations/group
- PATCH /conversations/:conversationId/preferences
- POST /conversations/:conversationId/leave
- PATCH /conversations/:conversationId/group
- POST /conversations/:conversationId/group/avatar
- POST /conversations/:conversationId/members
- DELETE /conversations/:conversationId/members/:memberId

Messages:

- GET /messages/search
- GET /messages/conversation/:conversationId
- POST /messages
- POST /messages/image
- POST /messages/:id/forward
- PATCH /messages/:id
- DELETE /messages/:id
- POST /messages/conversation/:conversationId/delivered
- POST /messages/conversation/:conversationId/read

Socket.io Overview

Socket authentication uses the same JWT identity as the REST API. The socket layer keeps users in stable user rooms and conversation rooms.

Realtime responsibilities:

- Register active socket connections.
- Join user-specific room.
- Join conversation rooms.
- Broadcast message-created events.
- Broadcast message-edited/deleted events.
- Broadcast delivered/read receipt events.
- Broadcast typing events.
- Broadcast online/offline presence changes.

Socket manager classes:

- ConnectionManager tracks active socket sessions.
- RoomManager builds and manages stable room names.
- Message, presence, and user handlers keep socket event logic separated.

Database Design

The canonical database schema is:

‘‘‘text
backend/database/schema.sql
‘‘‘

Main tables:

- users: registered users, profile avatar, status, last seen.
- conversations: direct/group conversation records.
- conversation_members: membership, role, archive/mute/pin/delete state, last read cursor.
- messages: hot/active messages.
- message_receipts: delivered/read state per user.
- attachments: uploaded message file metadata.
- messages_archive: partitioned archive table for old messages.
- device_tokens: future push notification device registration.

Important Database Strategies

- Conversations and members are separate tables, instead of deriving conversations only from messages.
- Group read state is per user through message_receipts and conversation_members.last_read_message_id.
- Messages use client_message_id for idempotent sends and retry safety.
- Recent/hot messages stay in messages.
- Old messages can be moved to messages_archive.
- Archive table is partitioned by time range.
- Frequently queried fields have indexes for cursor pagination, lookup, and membership checks.

Indexing Strategy

Important indexes include:

- users.username and users.email unique indexes.
- conversation_members(user_id).
- conversation_members(conversation_id).
- messages(conversation_id, message_id) for cursor pagination.
- messages(sender_id).
- messages(receiver_id).
- messages(reply_to_message_id).
- messages(forwarded_from_message_id).
- message_receipts(conversation_id, user_id).
- device_tokens(user_id, is_active).
- Archive indexes for conversation cursor and created time.

Message Lifecycle

Sending a Message

1. User writes a message in the React Native composer.
2. Frontend creates a clientMessageId.
3. UI adds an optimistic pending message.
4. Frontend sends REST request to backend.
5. Backend validates auth, payload, conversation membership, and business rules.
6. Backend stores the message in MySQL.
7. Backend creates receipt rows for conversation members.
8. Backend updates conversation last_message_id.
9. Backend invalidates affected Redis caches.
10. Backend emits Socket.io events.
11. Frontend replaces optimistic message with persisted message.

Reading a Conversation

1. Chat screen requests recent messages.
2. Backend checks recent message cache when eligible.
3. Backend loads messages from MySQL if cache misses.
4. Backend batch-loads receipts and attachments to avoid N+1 queries.
5. Backend returns cursor-based page.
6. Frontend marks conversation delivered/read when appropriate.
7. Backend updates read state only if there is an actual state change.

Replying

Reply metadata is stored in:

‘‘‘text
messages.reply_to_message_id
‘‘‘

The backend validates that the replied message belongs to the same conversation.

Forwarding

Forward metadata is stored in:

‘‘‘text
messages.forwarded_from_message_id
‘‘‘

The frontend supports selecting multiple forward targets and sends a forward request for each selected conversation/user.

Redis and Caching

Redis is used for:

- Recent message cache
- Conversation list cache
- User profile/status cache
- Presence state
- Rate limiting support
- Temporary realtime state patterns

Recent messages are cached by conversation and limit. Cache TTL is configurable through:

```text
RECENT_MESSAGES_CACHE_TTL
```

Default recent message cache:

```text
30 messages
60 seconds
```

Read/delivered invalidation is optimized to avoid clearing caches when nothing actually changed.

If Redis is unavailable, the backend has an in-memory fallback for development behavior.

Message Scalability Strategy

The app uses a hot-table plus archive-table strategy:

- messages stores active/recent messages.
- messages_archive stores old messages with receipt/attachment JSON snapshots.
- Archive table is partitioned by month.
- archiveOldMessages.js moves old messages in batches.
- Message listing can read across hot and archived storage.

This keeps recent chat access fast while still preserving old history.

Security

Current security measures:

- Passwords hashed with bcrypt.
- JWT access tokens for API/socket authentication.
- Refresh token flow.
- Auth middleware on protected routes.
- Request validation with express-validator.
- Centralized error handling.

- CORS configuration.
- Request sanitization middleware.
- Rate limiting.
- Authorization checks for profile updates and conversation/group membership.
- Group owner-only controls for sensitive group actions.
- Upload middleware separates avatar, group avatar, and message image upload paths.

Recommended production hardening:

- Store secrets in a real secret manager.
- Use HTTPS everywhere.
- Lock CORS to trusted origins.
- Add refresh token rotation/persistence.
- Move uploads to object storage such as S3/R2/GCS.
- Add antivirus/file scanning for uploads.
- Add audit logs for admin/group owner actions.
- Add database backups and restore testing.
- Add structured application monitoring.

Performance

Implemented performance strategies:

- Redis cache for frequently accessed recent messages.
- Cursor-based pagination instead of page numbers.
- Batch loading of receipts and attachments to avoid N+1 queries.
- Indexed database access paths.
- Conversation cache invalidation only when needed.
- Read/delivered state updates skip unnecessary writes.
- Hot/archive message split for large message volume.
- Optimistic frontend updates for perceived speed.

Future performance improvements:

- Direct-to-object-storage uploads with signed URLs.
- CDN for attachments and avatars.
- Redis adapter for Socket.io horizontal scaling.
- Read replicas for heavy read traffic.
- Background workers for notification fanout and media processing.
- Search engine integration for large-scale message search.

Business Logic Highlights

- A direct conversation is reused instead of creating duplicates.
- Group conversations require a title and at least one selected member.
- Group creators are stored as owners.
- Owners can edit group profile and manage members.
- Users can pin/mute/archive/delete conversations per membership row.

- Message retry uses clientMessageId to avoid duplicate persisted messages.
- A successful socket emit is not treated as durable persistence; messages are saved first, then emitted.
- Read state is tracked per user, which supports group chat correctness.
- Notifications respect mute settings on the frontend.

Localization

The app supports:

- English
- Myanmar

Static UI text is localized through LocalizationContext. Dynamic user content such as usernames, messages, group names, and emails is not translated.

Theme System

The app supports:

- Light mode
- Dark mode

Theme state is stored locally and applied through ThemeContext. Shared colors live in frontend/src/styles/colors.js.

Testing

Run backend tests:

‘‘‘bash
cd backend
npm test
‘‘‘

Covered backend areas include:

- Auth registration/login behavior
- Message sending
- Reply metadata validation
- Forward message behavior
- Read-state optimization
- Archived message serialization
- Batch-loading message relations
- Socket room and connection managers

Development Notes

Backend Port Already In Use

If port 5000 is already used, stop the old backend process or change PORT in backend/.env.

Metro Cache Issues

If Expo behaves strangely after package changes:

```bash
cd frontend
npm start -- --clear
```

Android Emulator Networking

If the emulator cannot reach the backend, set frontend env URLs to:

```text
http://10.0.2.2:5000
```

Physical Phone Networking

Use your computer LAN IP and make sure phone and computer are on the same network.

Current Limitations

- Push notification provider integration is prepared but not fully connected to FCM/APNs yet.
- Large file uploads currently go through Node/local storage; production should use object storage and signed URLs.
- Social login buttons are UI placeholders.
- MyDay/story publishing is a placeholder for future 24-hour status support.
- End-to-end encryption is not implemented.

License

This project is currently private/internal. Add a license before publishing publicly if needed.